`super()` Keyword in Python

- `super()` is a built-in function used to call the **parent class's methods or constructor**.
- It is mostly used in **inheritance** to reuse code from the parent class.

Why use `super()`

- To reuse parent class code (constructor or methods)
- To maintain clean and DRY (Don't Repeat Yourself) code
- Helps in multiple inheritance by following Method Resolution Order (MRO)

Example 1: Using `super()` to Call Parent Constructor

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)  # calls Person's
constructor
        self.student_id = student_id
```

Example 2: Calling Parent Method Using `super()`

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show_info(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, student_id):
        super().__init__(name, age)
        self.student_id = student_id

    def show_info(self):
        super().show_info()  # call parent method
        print(f"Student ID: {self.student_id}")
```

Key Points:

- super() automatically refers to the immediate parent class.
- You can use super() to call any method, not just __init__.
- Works well with method overriding — lets you extend parent functionality.

Magic Dunder Methods:

Magic Dunder Methods in Python (also known as **"dunder methods"** because of the double underscores before and after their names like `__init__`, `__str__` are **special methods** that allow you to define how your objects behave with built-in Python operations.

Here's a structured list with common dunder methods and their uses:

1. Object Initialization & Representation

Method	Purpose	Example
`__init__(self, ...)`	Constructor	Called when object is created
`__new__(cls, ...)`	Creates a new instance (used rarely)	Before `__init__`
`__str__(self)`	User-friendly string (used in `print()`)	`print(obj)`
`__repr__(self)`	Official string, used in debugging	`repr(obj)`

Example:

```
class Person:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"My name is {self.name}"

    def __len__(self):
        return len(self.name)

p = Person("Neeraj")
print(p)          # My name is Neeraj
print(len(p))     # 6
```

Operator Overloading in Python

Operator Overloading means giving **custom meaning to operators (+, -, , etc.)* for **user-defined objects** (i.e., class instances).

Python allows you to **override special methods** (also called **dunder methods**) like `__add__`, `__sub__`, `__mul__`, etc., to **define behavior of operators** for your objects.

Why Use It?

To perform **intuitive operations** on custom objects, like:

```
obj1 + obj2   # Instead of calling obj1.add(obj2)
```

Common Dunder Methods for Operator Overloading

Operator	Method	Example
+	`__add__`	`obj1 + obj2`
-	`__sub__`	`obj1 - obj2`
*	`__mul__`	`obj1 * obj2`
/	`__truediv__`	`obj1 / obj2`
==	`__eq__`	`obj1 == obj2`
>	`__gt__`	`obj1 > obj2`
<	`__lt__`	`obj1 < obj2`

Example 1: Operator Overloading with +

```
class Point:
  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __add__(self, other): # Overloading +
    return Point(self.x + other.x, self.y + other.y)

  def __str__(self):
    return f"({self.x},{self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)
p3 = p1 + p2 # This calls p1.__add__(p2)
print(p3)  # Output: (6, 8)
```

Example 2: Overloading == Operator

```
class Student:
```

```
  def __init__(self, name, marks):
    self.name = name
    self.marks = marks

  def __eq__(self, other): # Overloading ==
    return self.marks == other.marks

s1 = Student("Neeraj", 90)
s2 = Student("Amit", 90)

print(s1 == s2) # Output: True
```

Example 3: Overloading > Operator

```
class Box:
  def __init__(self, volume):
    self.volume = volume

  def __gt__(self, other): # Overloading >
    return self.volume > other.volume

b1 = Box(100)
b2 = Box(80)

print(b1 > b2) # Output: True
```

Important Points:

- Operator overloading is done using **dunder (double underscore) methods**.
- The method is automatically called when the corresponding operator is used.
- Both **objects involved** should be of compatible types.

Single Level Inheritance in Python

Single Level Inheritance means that **a child class inherits from a single parent class**.

Child class gets access to all the **methods and properties** of the parent class.

Example 1: With Constructor and `super()`

```
class Person:
  def __init__(self, name):
    self.name = name
    print(f"Person created: {self.name}")
```

```
class Student(Person):
  def __init__(self, name, roll):
    super().__init__(name) # Call parent constructor
    self.roll = roll
    print(f"Student created: Roll No {self.roll}")

s = Student("Neeraj", 101)
```

Output:

```
Person created: Neeraj
Student created: Roll No 101
```

Key Points:

- In single level inheritance:
- One **parent** → One **child**
- Child can **override** parent methods.
- Use `super()` to call parent constructor or methods.
- Helps in **code reusability**.

Diagram (Text Form):

```
     Parent
        ↑
      Child
```

Use Cases:

- Reuse common functionality (e.g., login system, user info).
- Add specific behaviour in child class (e.g., student, employee).

What is Multiple Inheritance?

Multiple inheritance means a class can **inherit from more than one parent class**.

Example:

```
class Father:
    def skills(self):
        print("Father: Cooking")

class Mother:
    def skills(self):
        print("Mother: Painting")

class Child(Father, Mother):
    def skills(self):
        print("Child: ", end="")
        super().skills()
# Will call Father's skills() because Father is listed first

obj = Child()
obj.skills()
```

Output:
```
Child: Cooking
```

Important Notes:

- Python **resolves conflicts using MRO** (Method Resolution Order).
- The method of the first parent (from left to right) is called if there is a name conflict.
- To see MRO of a class, use:

```
print(Child.__mro__)
```

Using All Parents' Methods:
If you want to call methods from **all parent classes**, do it explicitly:

```
class Child(Father, Mother):
    def skills(self):
        Father.skills(self)
        Mother.skills(self)
```

Advantages:

- Allows **code reuse** from multiple sources.
- Good for combining features from multiple classes.

Disadvantages:

- Can become **confusing** when multiple parents have the **same method name** (conflict).
- Can lead to **complex dependency structure**.

What is Multilevel Inheritance?

Multilevel inheritance means that a class **inherits from a child class**, which in turn **inherits from another parent class**.

It creates a **chain of inheritance** like:
Grandparent → Parent → Child

Example:

```
class Grandfather:
    def show_grandfather(self):
        print("I am the Grandfather.")

class Father(Grandfather):
    def show_father(self):
        print("I am the Father.")

class Son(Father):
    def show_son(self):
        print("I am the Son.")

# Create object of the last class in the chain
obj = Son()

# Access all methods
```

```
obj.show_grandfather()
obj.show_father()
obj.show_son()
```

Output:

```
I am the Grandfather.
I am the Father.
I am the Son.
```

Key Points:

- The child class gets access to **all features** of parent and grandparent classes.
- It promotes **code reuse** across multiple levels.
- Python allows any number of levels in the inheritance chain.

Advantages:

- Helps in building a clear hierarchy.
- Promotes **step-by-step inheritance** of features.

Disadvantages:

- If the chain becomes too long, it can become **complex to manage**.
- Changes in the top-level class may affect all subclasses.

What is Hybrid Inheritance?

Hybrid inheritance is a combination of two or more types of inheritance (like single, multiple, multilevel, hierarchical) in a single program.

It creates a **complex structure** where a class can inherit from multiple classes in different ways.

Example Structure:
Imagine this:

```
     A
    / \
   B   C
    \ /
     D
```

Here:

- B and C inherit from A (Hierarchical)
- D inherits from both B and C (Multiple)

This combination is known as **Hybrid Inheritance**.

Code Example:

```
class A:
    def feature_a(self):
        print("Feature A")
```

```python
class B(A):
    def feature_b(self):
        print("Feature B")

class C(A):
    def feature_c(self):
        print("Feature C")

class D(B, C):  # Inherits from both B and C
    def feature_d(self):
        print("Feature D")

# Create object of class D
obj = D()

# Accessing features from all parent classes
obj.feature_a()
obj.feature_b()
obj.feature_c()
obj.feature_d()
```

Output:

```
Feature A
Feature B
Feature C
Feature D
```

Key Concepts:

- Python uses **MRO (Method Resolution Order)** to resolve conflicts in Hybrid Inheritance.
- You can check the order using:

```
print(D.__mro__)
```

Advantages:

- Allows flexible and **powerful class design**.
- Enables combining multiple features from different class hierarchies.

Disadvantages:

- Can be **difficult to manage and understand**, especially if class names or method names conflict.
- Increases **complexity** of the codebase.

Summary:

- Hybrid Inheritance = **Mix of multiple inheritance types**.
- Use it carefully to avoid confusion.
- Python handles it smartly using MRO (left-to-right order).